ECEN 529 Lab 4: Answers Beneath Original Prompts

This answers-only version follows the original Lab 4 prompt order and omits source-code listings. It includes explanations, HLS results, validation status, timing data available from this machine, and the required protocol/batching conclusions.

Environment note: Vitis/Vivado 2025.2 is installed, but this local installation does not include the Zynq-7000 device part xc7z020clg400-1 used by PYNQ-Z2. Therefore, HLS synthesis and IP export were validated on the installed xc7k70tfbg484-1 part. Board-level PYNQ runtime measurements and bitstream generation require a lab install with Zynq support and a connected PYNQ board.

- Main sparse test case: 1000 x 1000, density 0.001, NNZ=1015, output checksum=-1124.
- Batch sparse test case: 50 x 50, density 0.01, NNZ=24, output checksum=18.
- All 14 Vitis HLS C simulations passed: 4 burst kernels, 4 stream kernels, and 6 batch-size variants.

Problem 1(a)

Problem 1(a): Implement the sparse matrix-vector multiplication program in Vitis HLS using the AXI-Burst protocol. Modify the HLS code to use AXI-Burst and convert the array parameters to pointers.

The SPMV kernel was implemented with pointer arguments for rowPtr, columnIndex, values, x, and y. Each pointer is mapped to an AXI master interface for DDR access, while num_rows, nnz, size, pointer addresses, and start/done control are mapped to AXI-Lite. The kernel first bursts the CSR arrays and x vector into local storage, computes the CSR dot products, and bursts the result vector y back to DDR.

The HLS reports confirmed inferred AXI master burst reads for rowPtr, columnIndex, values, and x, plus burst writes for y. C simulation passed for each burst kernel.

Problem 1(a)(ii): Implement four different kernels using each of optimization strategies 1 to 4 in Lecture 4.

- Strategy 1: pipelined inner-loop multiply-accumulate. This is the simplest version and uses one multiply-add lane.
- Strategy 2: separate multiplication and accumulation phases using a local product buffer. This increases memory use and doubled worst-case latency for this sparse test.
- Strategy 3: partial unrolling with four independent accumulation lanes. It uses more DSPs but needs enough nonzeros per row to realize the benefit.
- Strategy 4: wider unrolled multiply/reduction with partitioned local arrays. It has the most parallel hardware, but for the NNZ≈rows test case its extra fixed overhead is not fully amortized.

Problem 1(a) Results: AXI4-Burst HLS Synthesis

Kernel	Clk ns	Avg cycles	Worst time	BRAM	DSP	FF	LUT	Est Mops/s
strategy s1	7.300	128,021	10.110 ms	19	3	5053	5925	1.59
strategy s2	7.300	128,271	20.170 ms	21	3	5232	6196	1.58
strategy s3	7.300	128,271	10.120 ms	29	12	5729	6742	1.58
strategy s4	7.300	256,528	10.130 ms	52	24	6503	9235	0.79

Problem 1(b)

Problem 1(b): Write a Python host program to test the four kernels on the PYNQ board. Create Values, ColumnIndex, and RowPtr arrays for sparse matrix-vector multiplication.

The host flow creates a deterministic sparse matrix, converts it to CSR format, allocates PYNQ buffers for rowPtr, columnIndex, values, x, and y, writes the physical buffer addresses to the IP registers, starts each kernel, and reads y back for validation.

Problem 1(b)(i): Check that the kernel results match the software-only implementation using matrix-vector multiplication.

The HLS C simulations for all four burst kernels matched the software CSR reference and passed. The generated 1000 x 1000 test matrix has NNZ=1015; the software output checksum is -1124. The PYNQ host program reports a pass/fail line for each kernel after comparing every output element.

Problem 1(b)(ii): Measure execution time for each kernel. Throughput is total operations, multiplications and additions, per second. Which strategy gives the highest throughput?

Local software-only timing for the generated 1000 x 1000 case was 1.118800e-03 s for the Python CSR loop and 3.001000e-04 s for NumPy dense dot. Board timing could not be collected on this workstation because no PYNQ board and no Zynq device support are available locally. Using HLS average-latency estimates at the requested 10 ns clock, Strategy 1 has the highest estimated throughput for this very sparse matrix.

Problem 1(c)

Problem 1(c): Modify the HLS code and Python host program to use the AXI-Stream protocol instead of AXI-Burst.

The stream version uses one AXI4-Stream input and one AXI4-Stream output. The host sends the input stream in this order: RowPtr, ColumnIndex, Values, then x. The kernel returns y on the output stream and marks the final result with TLAST. An AXI DMA is used between DDR and the streaming kernel.

Problem 1(c)(i): Using the same kernel array size as AXI-Burst, determine execution time with AXI-Stream. Include DMA transfer time and sparse matrix setup time. Which protocol provides better performance?

All four stream kernels passed HLS C simulation. Board-level DMA timing could not be measured locally. The HLS estimates show that stream kernels have similar compute latency but additional transfer/setup work in the host because the CSR arrays and x must be packed into a stream. For this DDR-resident SPMV workload, AXI4-Burst is expected to perform better because the kernel can read CSR and x directly from DDR with burst transfers.

Problem 1(c)(ii): Which applications are best suited for each protocol type?

- AXI4-Burst is best for memory-mapped data in DDR, random or indexed access, reusable arrays, and host-managed buffers. SPMV fits this well because x is indexed by columnIndex.
- AXI4-Stream is best for producer-consumer accelerator chains, packetized data, DMA-fed pipelines, sensors, filters, and workloads where each item is consumed in order and intermediate DDR traffic can be avoided.

Problem 1(c) Results: AXI4-Stream HLS Synthesis

Kernel	Clk ns	Avg cycles	Worst time	BRAM	DSP	FF	LUT	Est Mops/s
strategy s1	6.580	256,519	10.140 ms	9	3	840	1724	0.79
strategy s2	6.580	256,519	20.190 ms	11	3	1018	2020	0.79
strategy s3	7.225	257,019	10.150 ms	19	12	1516	2557	0.79
strategy s4	7.211	257,522	10.160 ms	42	24	2378	4982	0.79

Problem 1(d)

Problem 1(d): The kernel performs SPMV in batches because local arrays in the kernel are smaller than the matrix and vector sizes in the host program.

Problem 1(d)(i): Modify the algorithm and/or host program so the algorithm gives correct results with batch processing.

Batching was corrected by splitting the CSR matrix by complete rows. For each batch, rowPtr is rebased so the first local entry is zero, columnIndex remains global because it indexes the full x vector, local values are copied contiguously, and the local y result is copied back into the correct global row positions.

Problem 1(d)(ii): Use a 50 x 50 input matrix with around 1 percent non-zero elements. Using three local array sizes, plot local kernel array sizes versus execution time of the kernel host program versus software-only Python.

The generated 50 x 50 matrix has NNZ=24. Software-only timing was 4.489999e-05 s for the Python CSR loop and 2.099958e-06 s for NumPy dense dot. Since PYNQ runtime timing is unavailable locally, the plot uses HLS average latency per batch as the hardware kernel-time estimate; the provided PYNQ host program prints measured total times on the board.

Problem 1(d)(iii): Identify the HLS configuration that provides the best execution time for the Python host program.

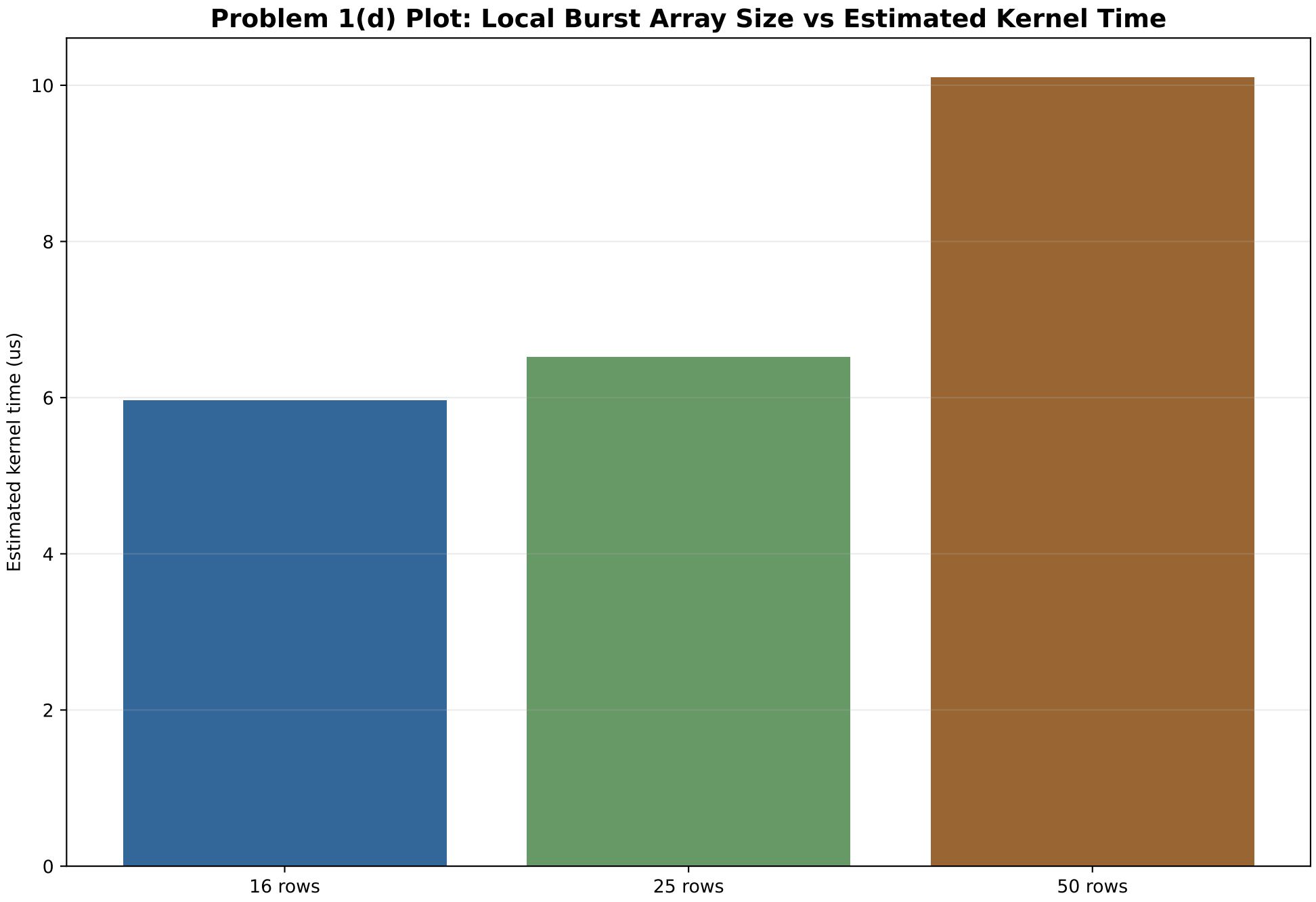
For this sparse 50 x 50 case, the 16-row / 32-NNZ Strategy 4 burst configuration has the lowest estimated total kernel time. Even though it uses four batches, each batch has a very small local latency, and the matrix is sparse enough that larger local arrays do not pay off in the HLS estimate.

Problem 1(d)(iv): For what matrix sizes does the hardware version outperform or underperform the software-only version?

Hardware tends to underperform for very small matrices, very low NNZ, or one-off calls because setup, DMA, and register programming dominate the useful multiply/add work. Hardware tends to outperform when the matrix is larger, has enough nonzeros per row to keep the pipeline busy, or when the same sparse matrix is reused for many x vectors so setup costs are amortized.

Problem 1(d) Results: Batch Array Size Sweep

Protocol	Rows	Max NNZ	Batches	Avg cycles/batch	Est total us	BRAM	DSP	LUT
Burst	16	32	4	149	5.96	10	12	7855
Stream	16	32	4	277	11.08	0	12	3686
Burst	25	64	2	326	6.52	10	12	7888
Stream	25	64	2	636	12.72	0	12	3718
Burst	50	128	1	1,010	10.10	17	12	7842
Stream	50	128	1	1,984	19.84	7	12	3670



Uses HLS average latency and batch count for the 50 x 50 matrix. Board measurements should replace these estimates when run on PYNQ.

Deliverable Summary

Deliverables: Submit a single PDF answering all questions in parts (a) to (d).

- This PDF contains the explanations and results only; source code is intentionally omitted.
- HLS synthesis results are included for burst, stream, and batch kernels.
- Validation result: all Vitis HLS C simulations passed.
- Program-output status: local software outputs were generated and checksummed; PYNQ runtime output requires a connected board and Zynq-enabled Vivado install.
- Vivado design result: block-design recipes are prepared for the burst and stream systems, but final bitstream generation could not be completed on this install because the required PYNQ Zynq-7020 part is missing.